CHAPTER 2

Logic representation and minimization technique

FY. Btech DLD

COMPUTERS AND ELECTRICITY

Gate

A device that performs a basic operation on electrical signals

Circuits

Gates combined to perform more complicated tasks

COMPUTERS AND ELECTRICITY

How do we describe the behavior of gates and circuits?

Boolean expressions

Uses Boolean algebra, a mathematical notation for expressing two-valued logic

Logic diagrams

A graphical representation of a circuit; each gate has its own symbol

Truth tables

A table showing all possible input value and the associated output values

GATES

Seven types of gates

- NOT
- AND
- OR
- XOR
- NAND
- NOR
- EXNOR

Typically, logic diagrams are black and white with gates distinguished only by their shape

We use color for emphasis (and fun)

NOT GATE

A NOT gate accepts one input signal (0 or 1) and returns the opposite signal as output

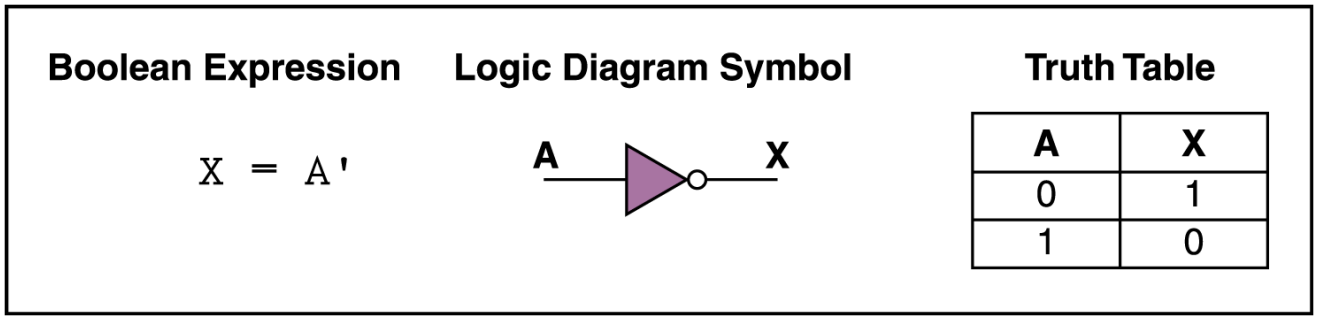


Figure 4.1 **Various representations of a NOT gate**

AND GATE

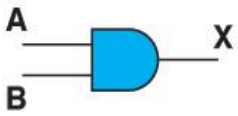
An AND gate accepts two input signals

If both are 1, the output is 1; otherwise, the output is 0

Boolean Expression

$X = A \cdot B$

Logic Diagram Symbol



Truth Table

A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

Figure 4.2 **Various representations of an AND gate**

AND GATE EXAMPLE

Example 1.1

You have rented a locker in a bank. Express the process of opening the locker in terms of a digital operation.

Solution

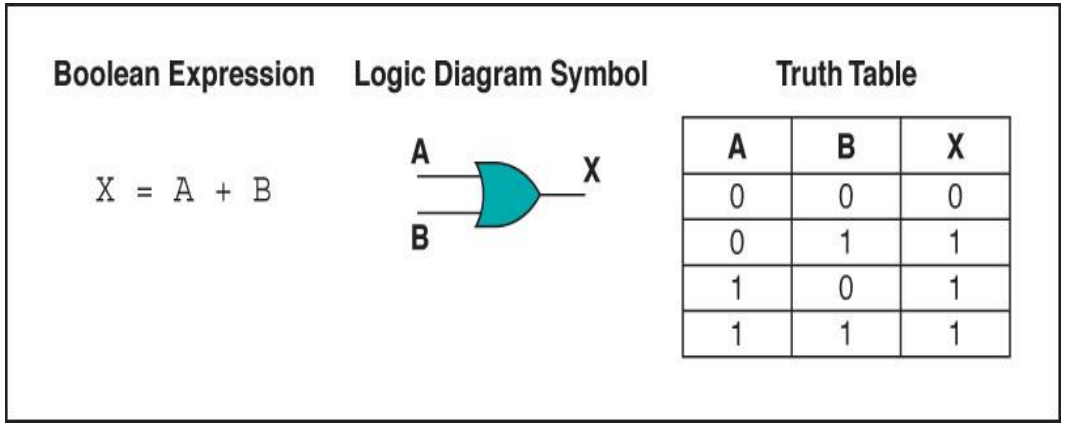
The locker door (Y) can be opened by using one key (A) which is with you and the other key (B) which is with the bank executive. When both the keys are used, the locker door opens, i.e., the locker door can be opened ($Y = 1$) only when both the keys are applied ($A = B = 1$). Thus, this process can be expressed as an AND operation

$$Y = A \cdot B$$

OR GATE

An OR gate accepts two input signals

If both are 0, the output is 0; otherwise, the output is 1



A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

Figure 4.3 **Various representations of a OR gate**

OR GATE EXAMPLE

Example 1.3

In a chemical process an ALARM is required to be activated if either temperature or pressure or both exceed certain limits. Is it possible to express this operation in terms of a digital operation? If yes, find the operation.

Solution

Let the temperature and pressure be converted into electrical signals and $T = 1$ if temperature exceeds the specified limit and $P = 1$ if pressure exceeds the specified limit. If $T = 1$ or $P = 1$ or both T and P are 1 then the ALARM is required to be activated, i.e., the signal applied to the ALARM $Y = 1$. This operation can be expressed as

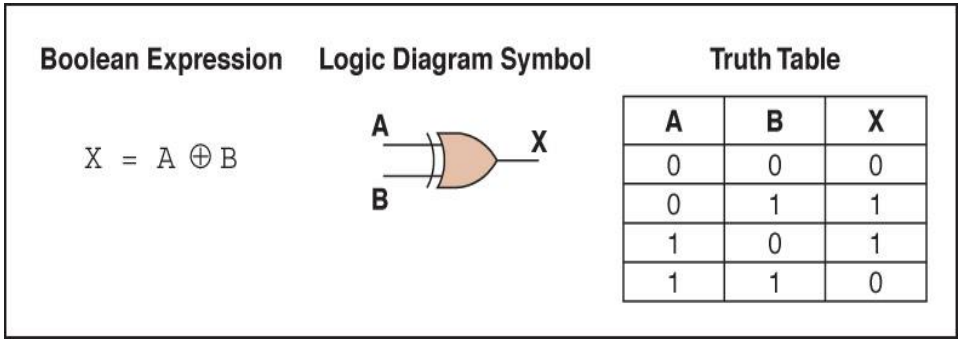
$$Y = T \text{ OR } P$$
$$= T + P$$

Which is an OR operation.

XOR GATE

An XOR gate accepts two input signals

If both are the same, the output is 0; otherwise, the output is 1



A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Figure 4.4 Various representations of an XOR gate

XOR GATE

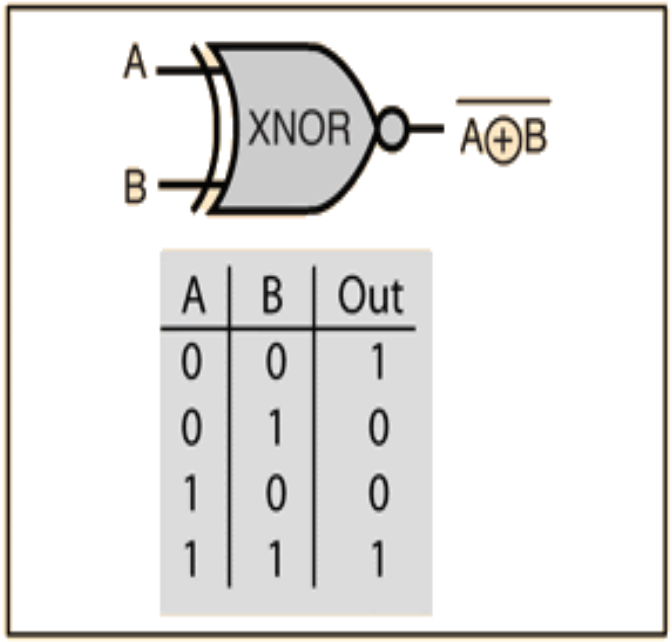
Note the difference between the XOR gate and the OR gate; they differ only in one input situation

When both input signals are 1, the OR gate produces a 1 and the XOR produces a 0

XOR is called the *exclusive OR*

EX-NOR GATE

- Like the EX-OR gate, the EX-NOR gate is not a basic operation and can be performed using basic or universal gates.
- It is referred to as the *coincidence* operation because it produces an output of 1 when its inputs coincide in value (both 0 or both 1)



A	B	Out
0	0	1
0	1	0
1	0	0
1	1	1

NAND GATE

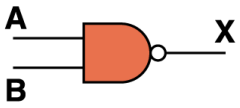
The NAND gate accepts two input signals

If both are 1, the output is 0; otherwise, the output is 1

Boolean Expression

$X = (A \cdot B)'$

Logic Diagram Symbol



Truth Table

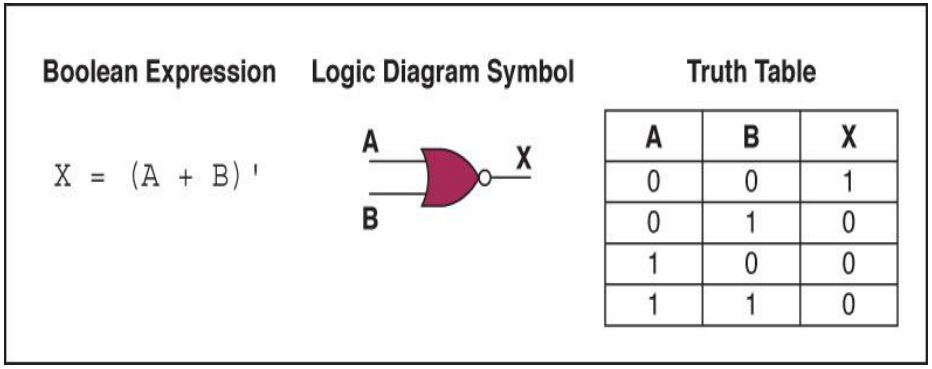
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

Figure 4.5 Various representations of a NAND gate

NOR GATE

The NOR gate accepts two input signals

If both are 0, the output is 1; otherwise, the output is 0



A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

Figure 4.6 **Various representations of a NOR gate**

DE MORGANS THEOREM

- We use De Morgan's theorems to solve the expressions of Boolean Algebra. It is a very powerful tool used in digital design.
- This theorem explains that the complements of the products of all the terms are equal to the sums of the complements of each and every term.
- Likewise, the complements of the sums of all the terms are equal to the products of the complements of each and every term.

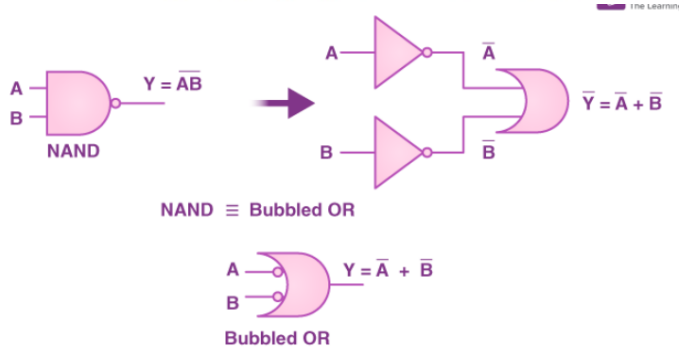
DE MORGAN'S FIRST THEOREM

Theorem 1

$$\overline{A.B} = \overline{A} + \overline{B}$$

NAND = Bubbled OR

- The LHS (left-hand side) of this theorem represents the NAND gate that has inputs A and B. On the other hand, the RHS (right-hand side) of this theorem represents the OR gate that has inverted inputs.
- The OR gate here is known as a Bubbled OR.



A	B	$\overline{A+B}$	$\overline{A}$	$\overline{B}$	$\overline{A}+\overline{B}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

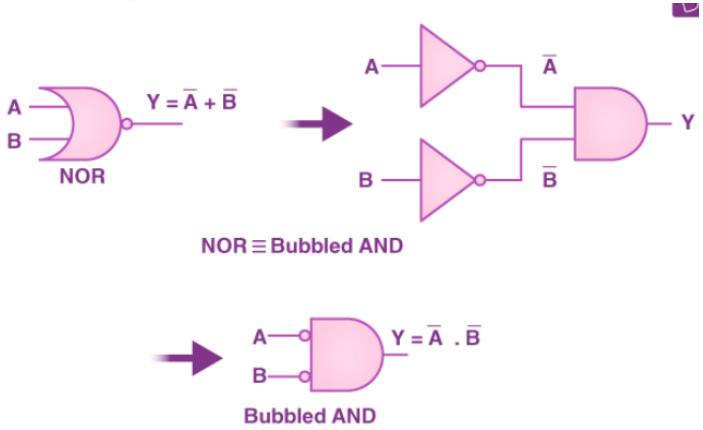
DE MORGAN'S SECOND THEOREM

Theorem 2

$$\overline{A + B} = \overline{A} . \overline{B}$$

NOR = Bubbled AND

- The left-hand side of this theorem represents the NOR gate that has inputs A and B. On the other hand, the right-hand side represents the AND gate that has inverted inputs.
- The AND gate here is known as a Bubbled AND.

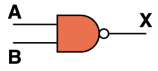


A	B	$\overline{A + B}$	$\overline{A}$	$\overline{B}$	$\overline{A} . \overline{B}$
0	0	1	1	1	1
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	0

UNIVERSAL GATES

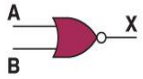
In Boolean Algebra, the NAND and NOR gates are called universal gates because any digital circuit can be implemented by using any one of these two i.e. any logic gate can be created using NAND or NOR gates only.

NAND GATE

Boolean Expression	Logic Diagram Symbol	Truth Table
$X = (A \cdot B)'$		

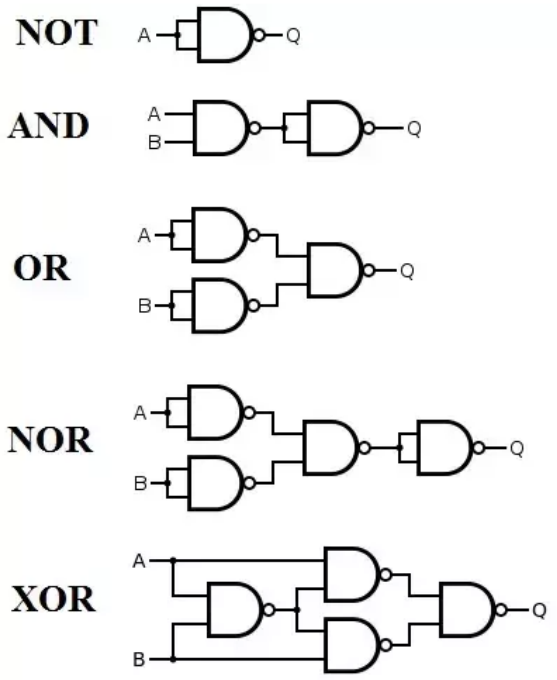
A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE

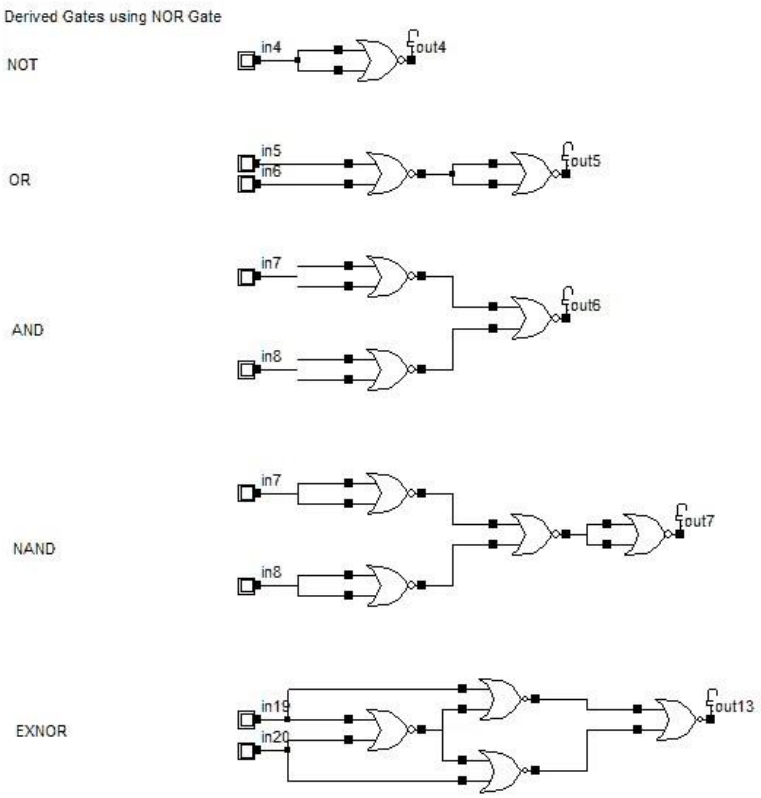
Boolean Expression	Logic Diagram Symbol	Truth Table
$X = (A + B)'$		

A	B	X
0	0	1
0	1	0
1	0	0
1	1	0

NAND AS A UNIVERSAL GATE



NOR AS A UNIVERSAL GATE



REVIEW OF GATE PROCESSING

A NOT gate inverts its single input

An AND gate produces 1 if both input values are 1

An OR gate produces 0 if both input values are 0

An XOR gate produces 0 if input values are the same

A NAND gate produces 0 if both inputs are 1

A NOR gate produces a 1 if both inputs are 0

GATES WITH MORE INPUTS

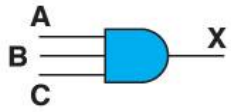
Gates can be designed to accept three or more input values

A three-input AND gate, for example, produces an output of 1 only if all input values are 1

Boolean Expression

$X = A \cdot B \cdot C$

Logic Diagram Symbol



Truth Table

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Figure 4.7 Various representations of a three-input AND gate

BOOLEAN ALGEBRA

- Digital signals are discrete in nature and can only assume one of the two values 0 or 1. A number system based on these two digits is known as binary number system.
- In the middle of 19th century, an English mathematician George Boole developed rules for manipulations of binary variables, known as Boolean algebra.
- This is the basis of all digital systems like computers, calculators, etc. Binary variables can be represented by a letter symbol such as A, B, X, Y,….. The variable can have only one of the two possible values at any time, viz. 0 or 1.
- From Boolean algebraic theorems, we observe that the even numbered theorems can be obtained from their preceding odd numbered theorems by (i) interchanging '+' and '.' signs, and (ii) interchanging 0 and 1.Theorems which are related in this way are called duals.

BOOLEAN ALGEBRAIC THEOREM

Table 1.8 ***Boolean Algebraic Theorems***

Theorem No.	Theorem
1.1	$A + 0 = A$
1.2	$A \cdot 1 = A$
1.3	$A + 1 = 1$
1.4	$A \cdot 0 = 0$
1.5	$A + A = A$
1.6	$A \cdot A = A$
1.7	$A + \overline{A} = 1$
1.8	$A \cdot \overline{A} = 0$
1.9	$A \cdot (B + C) = AB + AC$
1.10	$A + BC = (A + B)(A + C)$
1.11	$A + AB = A$

(*Continued*)

BOOLEAN ALGEBRAIC THEOREM (CONT..)

Theorem No.	Theorem
1.12	$A\,(A + B) = A$
1.13	$A + \overline{A}B = (A + B)$
1.14	$A(\overline{A} + B) = AB$
1.15	$AB + A\overline{B} = A$
1.16	$(A + B) \cdot (A + \overline{B}) = A$
1.17	$AB + \overline{A}C = (A + C)\,(\overline{A} + B)$
1.18	$(A + B)\,(\overline{A} + C) = AC + \overline{A}B$
1.19	$AB + \overline{A}C + BC = AB + \overline{A}C$
1.20	$(A + B)\,(\overline{A} + C)\,(B + C) = (A + B)\,(\overline{A} + C)$
1.21	$\overline{A \cdot B \cdot C \cdot \ldots} = \overline{A} + \overline{B} + \overline{C} + \ldots$
1.22	$\overline{A + B + C + \ldots} = \overline{A} \cdot \overline{B} \cdot \overline{C} \ldots$

PROPERTIES OF BOOLEAN ALGEBRA

Property	AND	OR
Commutative	AB = BA	A + B = B + A
Associative	(AB)C = A(BC)	(A + B) + C = A + (B + C)
Distributive	A(B + C) = (AB) + (AC)	A + (BC) = (A + B)(A + C)
Identity	A1 = A	A + 0 = A
Complement	A(A') = 0	A + (A') = 1
DeMorgan's law	(AB)' = A' OR B'	(A + B)' = A'B'

BOOLEAN ALGEBRA EXAMPLES

- Simplify: $C + \overline{B\overline{C}}$:

Expression	Rule(s) Used
$C + \overline{B\overline{C}}$	Original Expression
$C + (\overline{B} + \overline{\overline{C}})$	DeMorgan's Law.
$(C + \overline{\overline{C}}) + \overline{B}$	Commutative, Associative Laws.
$T + \overline{B}$	Complement Law.
T	Identity Law.

- Simplify: $\overline{A\overline{B}}(\overline{A} + B)(\overline{B} + B)$:

Expression	Rule(s) Used
$\overline{A\overline{B}}(\overline{A} + B)(\overline{B} + B)$	Original Expression
$\overline{A\overline{B}}(\overline{A} + B)$	Complement law, Identity law.
$(\overline{A} + \overline{\overline{B}})(\overline{A} + B)$	DeMorgan's Law
$\overline{A} + \overline{\overline{B}}B$	Distributive law. This step uses the fact that or distributes over
$\overline{A}$	Complement, Identity.

STANDARD REPRESENTATIONS FOR LOGIC FUNCTIONS

Logic functions are expressed in terms of logical variables.

The values assumed by the logic functions as well as the logic variables are in the binary form.

Any arbitrary logic function can be expressed in the following forms:

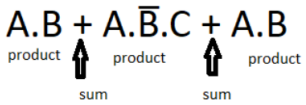
- **Sum-of-products form (SOP)**, and
- **Product-of-sums form (POS)**

This does not mean that the logic function cannot be written in any other form.

It can be written in various forms but the above two forms are conveniently suited in arriving at the standard methods for designing the circuits which will become clear from the following discussions.

SUM OF PRODUCT (SOP)

- The Sum of Product (SOP) form refers to a Boolean expression where several product terms (involving AND operations) are added up.
- Each individual term in SOP is also called as minterm.
- This is perhaps the most popular form used in the design and optimization of digital systems.
- In SOP format, each of those elements is a combination (ANDing) of some variables (or their negations); all of them are combined by means of disjunction (ORing).
- This is very helpful in, for instance, the creation of logic circuits from truth tables or achieving algorithmic simplifications of logic functions.
- Example :



SOP Form

SOP (CONT..)

The SOP form can be in either canonical form or non-canonical form.

1. Non-Canonical SOP Form

In this form each product term between may or may not contain all the variables of the function.

Example:

$F(A,B,C) = A + \bar{B}.C + A.C$

as we can see in above example the function have variables A, B, C but we are not including each variable in each product term.

SOP (CONT..)

2. Canonical SOP Form

In canonical SOP form each product term contains all the variables of the function, where variables in each product term can be in true form or complemented form.

Example :

$F(A,B) = \bar{A}.B + A.\bar{B}$

As we can see in above example each product term contain all the variables which are present in function.

The SOP is written in the format shown below called as short hand for K-map minimization technique

$F(A,B,C) = \sum m(2, 4, 5, 6, 7)$

PRODUCT OF SUM (POS)

- POS form is the product of all the sums.
- Each individual term in POS is also called maxterm.
- Here, the sum does not mean traditional addition, the sum here refers to the 'OR' operation, and the product here refers to the 'AND' operation.
- Thus, in POS form, we perform the OR of multiple variables and then perform the AND operation between them.
- Example: F = (X + Y + Z) . (X + Y + Z') . (X + Y' + Z')

POS (CONT..)

We have two types of POS form:

1.Canonical POS form

2.Minimal POS form.

In canonical POS form, we have all the variables present in a complimented or un-complimented form in each term.

Example: $F = (X + Y + Z) \cdot (X + Y + Z') \cdot (X + Y' + Z')$

In contrast, in minimal POS form, we do not have all the variables in the complimented or uncomplimented form.

Example: $F = (X + Y) \cdot (X + Z')$

The short hand form of POS is given below

$$Y = \Pi M(1, 2, 4, 5, 8, 9, 11, 13, 14).$$

RELATION BETWEEN SOP AND POS

- $\boldsymbol{Y = \Sigma m\,(3, 4, 6, 7)}$ ……. (1)
- $\boldsymbol{Y = \Pi M(0, 1, 2, 5)}$ ……. (2)
- Since these two equations represent the same logical function therefore we notice that there is a complementary type of relationship between a function expressed in terms of minterms and in terms of maxterms.
- Here, we are dealing with a three-variable function where the number of minterms and maxterms are $2^3 = 8$ and the corresponding decimal numbers are 0 through 7.
- Out of this the terms corresponding to decimal numbers 3, 4, 6, and 7 are minterms, and the terms corresponding to decimal numbers 0, 1, 2, and 5 are maxterms.

INTRODUCTION TO KARNAUGH MAPS (K-MAPS)

- A K-map is a graphical technique used for the systematic simplification and manipulation of Boolean expressions.
- Information from a truth table, SOP form, or POS form can be directly represented on the map.
- It is one of the most extensively used tools for simplifying Boolean functions because it is more intuitive than purely algebraic methods.
- Although the technique may be used for any number of variables, it is generally used up to six variables beyond which it becomes very difficult.

K-MAP STRUCTURE AND SCALING

- **Cell Count**: An n-variable K-map contains 2^n individual cells.
- **Cell Correspondence**: Each cell corresponds to one specific combination of the n variables.

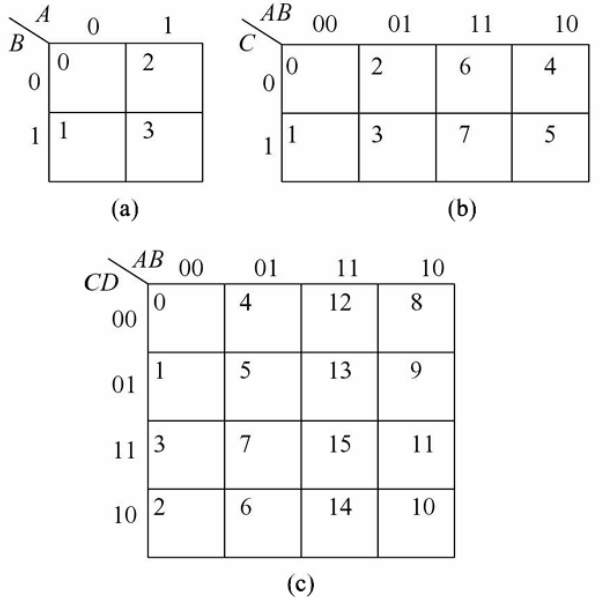
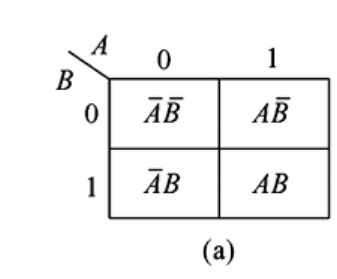


Fig. 5.5 ***Karnaugh-maps (a) Two-variable (b) Three-variable (c) Four-variable***

K-MAP FOR SOP



B \ A	0	1
0	$\bar{A}\bar{B}$	$A\bar{B}$
1	$\bar{A}B$	AB

(a)

2x2 K-map

(d)

CD \ AB	00	01	11	10
00	$\bar{A}\bar{B}\bar{C}\bar{D}$	$\bar{A}B\bar{C}\bar{D}$	$AB\bar{C}\bar{D}$	$A\bar{B}\bar{C}\bar{D}$
01	$\bar{A}\bar{B}\bar{C}D$	$\bar{A}B\bar{C}D$	$AB\bar{C}D$	$A\bar{B}\bar{C}D$
11	$\bar{A}\bar{B}CD$	$\bar{A}BCD$	$ABCD$	$A\bar{B}CD$
10	$\bar{A}\bar{B}C\bar{D}$	$\bar{A}BC\bar{D}$	$ABC\bar{D}$	$A\bar{B}C\bar{D}$

4x4 K-map

C \ AB	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$

3x3 K-map

PRIME IMPLICANTS

- In choosing adjacent squares in a map, we must ensure that

(1) all the minterms of the function are covered when we combine the squares,

(2) the number of terms in the expression is minimized, and

(3) there are no redundant terms (i.e., minterms already covered by other terms).

Sometimes there may be two or more expressions that satisfy the simplification criteria.

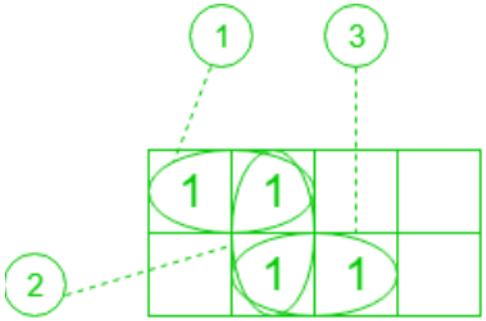
The procedure for combining squares in the map may be made more systematic if we understand the meaning of two special types of terms.

A prime implicant is a product term obtained by combining the maximum possible number of adjacent squares in the map. If a minterm in a square is covered by only one prime implicant, that prime implicant is said to be essential

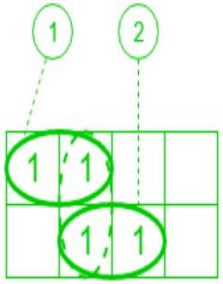
PRIME IMPLICANTS (CONT..)

- *The prime implicants of a function can be obtained from the map by combining all possible maximum numbers of squares.*
- This means that a single 1 on a map represents a prime implicant if it is not adjacent to any other 1's.
- Two adjacent 1's form a prime implicant, provided that they are not within a group of four adjacent squares.
- Four adjacent 1's form a prime implicant if they are not within a group of eight adjacent squares, and so on.
- *The essential prime implicants are found by looking at each square marked with a 1 and checking the number of prime implicants that cover it*.
- *The prime implicant is essential if it is the only prime implicant that covers the minterm.*

PRIME IMPLICANTS (CONT..)



No. of Prime Implicants = 3



No. of Essential Prime Implicants = 2

NUMERICAL

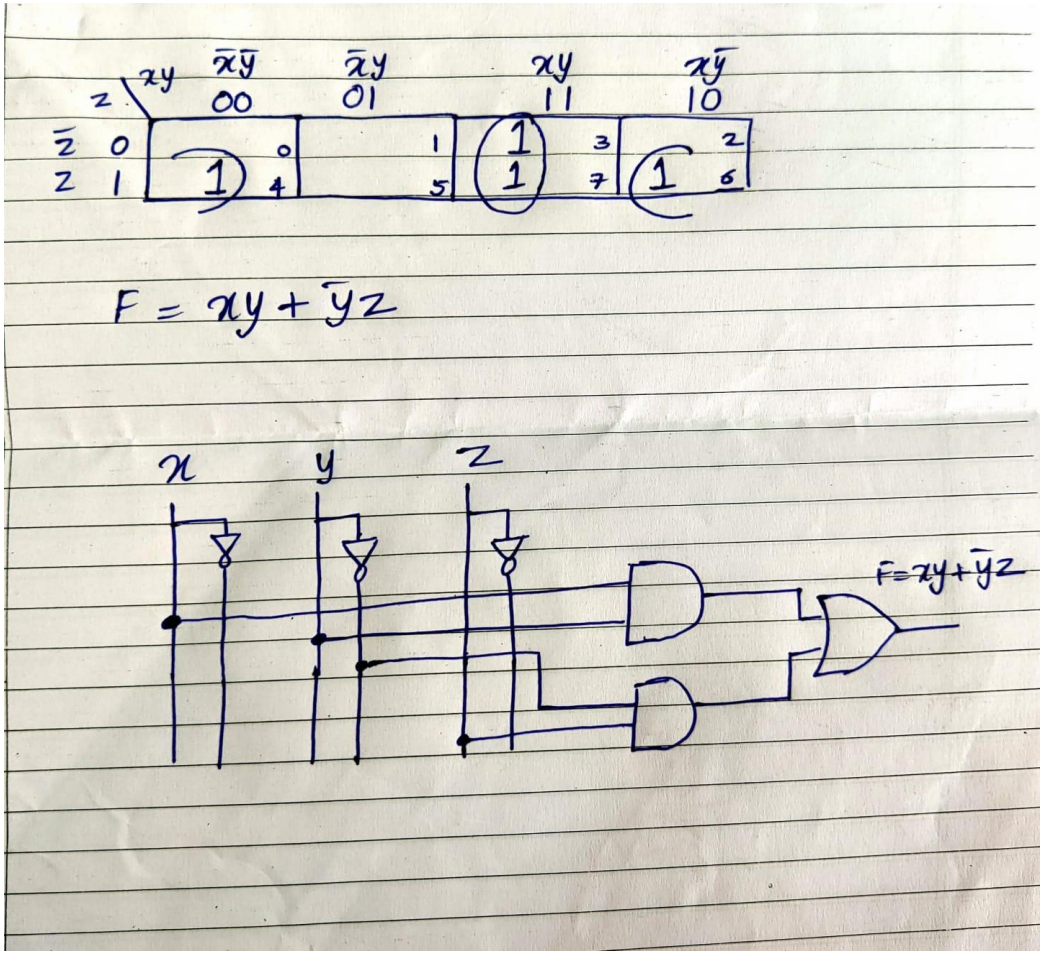
1st numerical:

Solve the following using K-map minimization technique for SOP form

$$F(x, y, z) = \Sigma m(3,4,6,7)$$

NUMERICAL

Solution to 1st numerical



NUMERICAL

2nd numerical:

Solve the following using K-map minimization technique for SOP form

$$F(A, B, C) = \Sigma m(2,3,4,5)$$

NUMERICAL

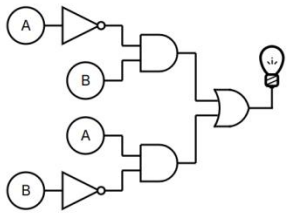
Solution to 2nd numerical

AB \ C	0	1
00	0 0	0 1
01	1 2	1 3
11	0 6	0 7
10	1 4	1 5

Simplified Boolean Expression

A'B + AB'

Logic Circuit



NUMERICAL

3rd numerical:

Solve the following using K-map minimization technique for SOP form

$$F(w, x, y, z) = \Sigma m(0,1,2,4,5,6,8,9,12,13,14)$$

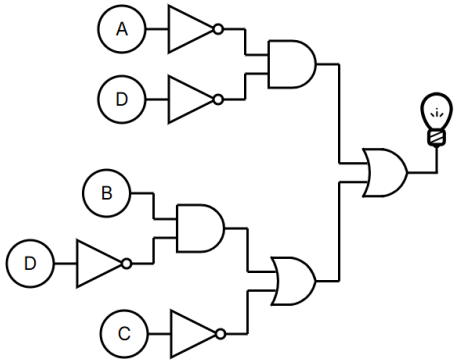
NUMERICAL

Solution to 3rd numerical

CD \ AB	00	01	11	10
00	1 (0)	1 (1)	0 (3)	1 (2)
01	1 (4)	1 (5)	0 (7)	1 (6)
11	1 (12)	1 (13)	0 (15)	1 (14)
10	1 (8)	1 (9)	0 (11)	0 (10)

Simplified Boolean Expression

A'D' + BD' + C'



NUMERICAL

4th numerical:

Solve the following using K-map minimization technique for SOP form

$$F(A, B, C, D) = \Sigma m(2,4,6,8,10,11)$$

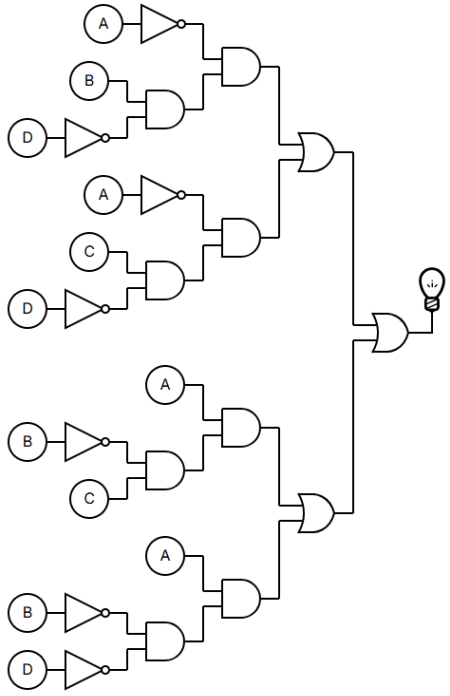
NUMERICAL

Solution to 4th numerical

CD \ AB	00	01	11	10
00	0 (0)	0 (1)	0 (3)	1 (2)
01	1 (4)	0 (5)	0 (7)	1 (6)
11	0 (12)	0 (13)	0 (15)	0 (14)
10	1 (8)	0 (9)	1 (11)	1 (10)

Simplified Boolean Expression

A'BD' + A'CD' + AB'C + AB'D'

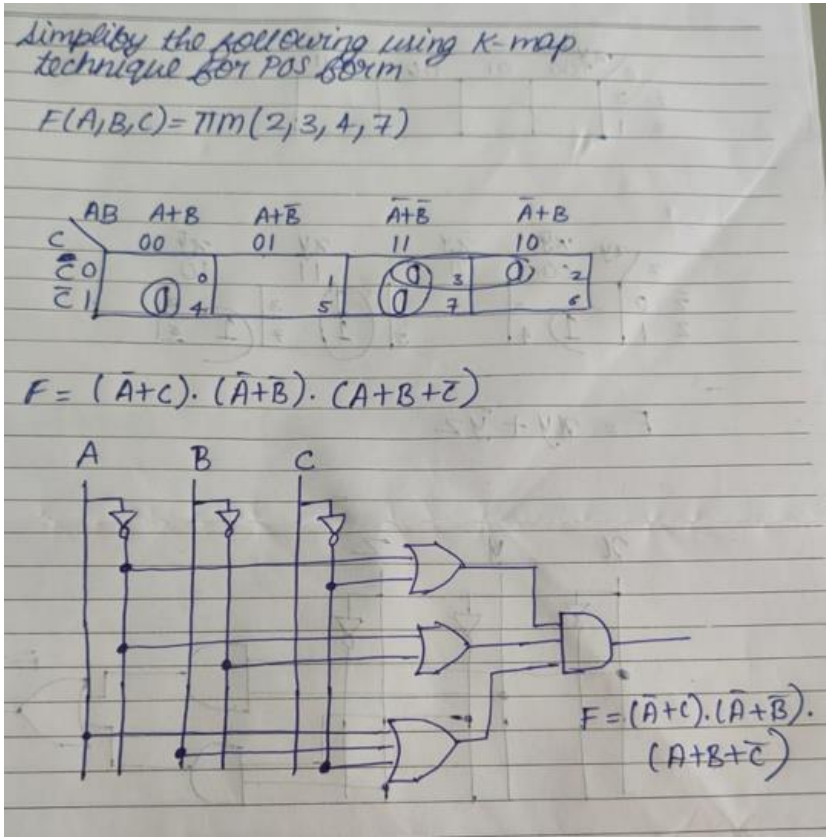


NUMERICAL ON POS FORM

Solve the following using K-map minimization technique for POS form

$$F(A, B, C, D) = \Pi M(2,3,4,7)$$

NUMERICAL ON POS FORM

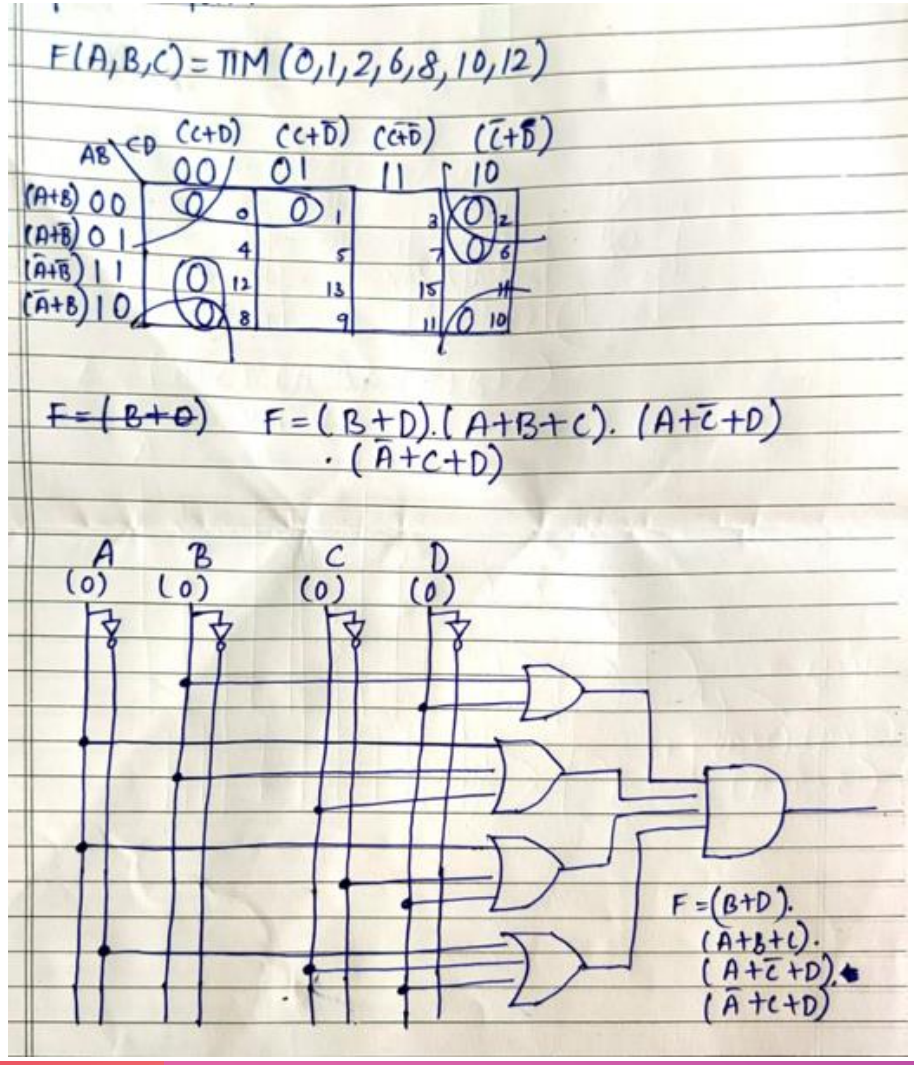


NUMERICAL ON POS FORM

Solve the following using K-map minimization technique for POS form

$$F(A, B, C, D) = \prod M(0,1,2,6,8,10,12)$$

NUMERICAL ON POS FORM



DON'T-CARE CONDITIONS

- *A don't care minterm is the is a combination of variables whose whose logical value is not specified*.
- Such a minterm cannot be marked with a '1' in the map because it will require the function to be always 1
- Likewise putting a zero will require the function to be always '0'.
- To distinguish don't-care condition from '1' and '0' an 'X' is used. Thus an 'X' inside the square in the map indicates that we don't care weather the value of 0 or 1 is assigned to F for a particular minterm.
- In choosing adjacent terms the don't-care terms can be assumed as zero or one.

NUMERICAL

Simplify the following function

F (w,x,y,z) = Σm (1,3,7,11,15)

Which has don't care condition : d(w,x,y,z) = Σ (0,2,5)

The implementation must be using only universal gates

SOLUTION

Somaiya TRUST

Batch: Roll No.:
Experiment / assignment / tutorial No.
Grade: AA / AB / BB / BC / CC / CD / DD
Signature of the Faculty In-Charge with date

Simplify the following function using SOP form

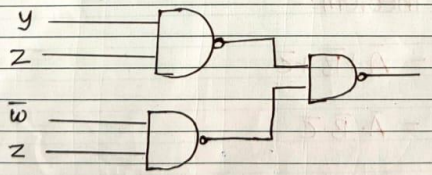
$F(w,x,y,z) = \Sigma m(1,3,7,11,15)$
$d(w,x,y,z) = \Sigma(0,2,5)$

Solution:-

Solving by 4-variable K-map for SOP form

wx \ yz	$\bar{y}\bar{z}$ 00	$\bar{y}z$ 01	yz 11	$y\bar{z}$ 10
$\bar{w}\bar{x}$ 00	X (0)	1 (1)	1 (3)	X (2)
$\bar{w}x$ 01	(4)	X (5)	1 (7)	(6)
wx 11	(12)	(13)	1 (15)	(14)
$w\bar{x}$ 10	(8)	(9)	1 (11)	(10)

$F = yz + \bar{w}z$



Implementation using NAND Gate.

K.J. Somaiya College of Engineering, Vidyavihar (E), Mumbai - 400 077.

NUMERICAL

Simplify the following function

F (w,x,y,z) = ΠM (2,3,4,7)

Which has don't care condition : d(w,x,y,z) = Σ (0,5)

Implementation of circuit must be using only universal gates

SOLUTION

Simplify the following using K-map technique for POS form

$F(A,B,C) = \Pi M(2,3,4,7)$
$d(A,B,C) = \Pi(0,5)$

Solution:-

Solving using three variable K-map

C \ AB	A+B (00)	A+$\bar{B}$ (01)	$\bar{A}$+$\bar{B}$ (11)	A+$\bar{B}$ (10)
$\bar{C}$ 0	X 0	1	0 3	0 2
C 1	0 4	X 5	0 7	6

$F = (\bar{B}+\bar{C}) \cdot (\bar{A}+\bar{B}) (A+B)$

Circuit implementation using only NOR Gate:-

De Morgans Theorem:-

$\overline{A \cdot B \cdot C} = \bar{A}+\bar{B}+\bar{C}$

$\overline{A+B+C} = \bar{A} \cdot \bar{B} \cdot \bar{C}$

$\bar{\bar{A}} = A$

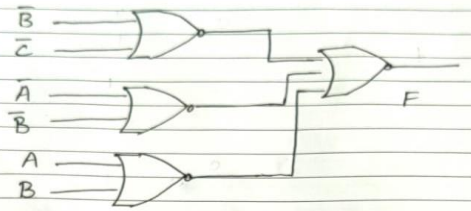
$F = \overline{(\bar{B}+\bar{C}) \cdot (\bar{A}+\bar{B}) \cdot (A+B)}$

$F = \overline{(\bar{B}+\bar{C})} + \overline{(\bar{A}+\bar{B})} + \overline{(A+B)}$

Somaya TRUST

Batch: Roll No.:
Experiment / assignment / tutorial No.
Grade: AA / AB / BB / BC / CC / CD / DD
Signature of the Faculty In-Charge with date

Circuit implementation:-



Truth table:-

Input	Output
0 0 0	X
0 0 1	0
0 1 0	1
0 1 1	1
1 0 0	1
1 0 1	X
1 1 0	0
1 1 1	1

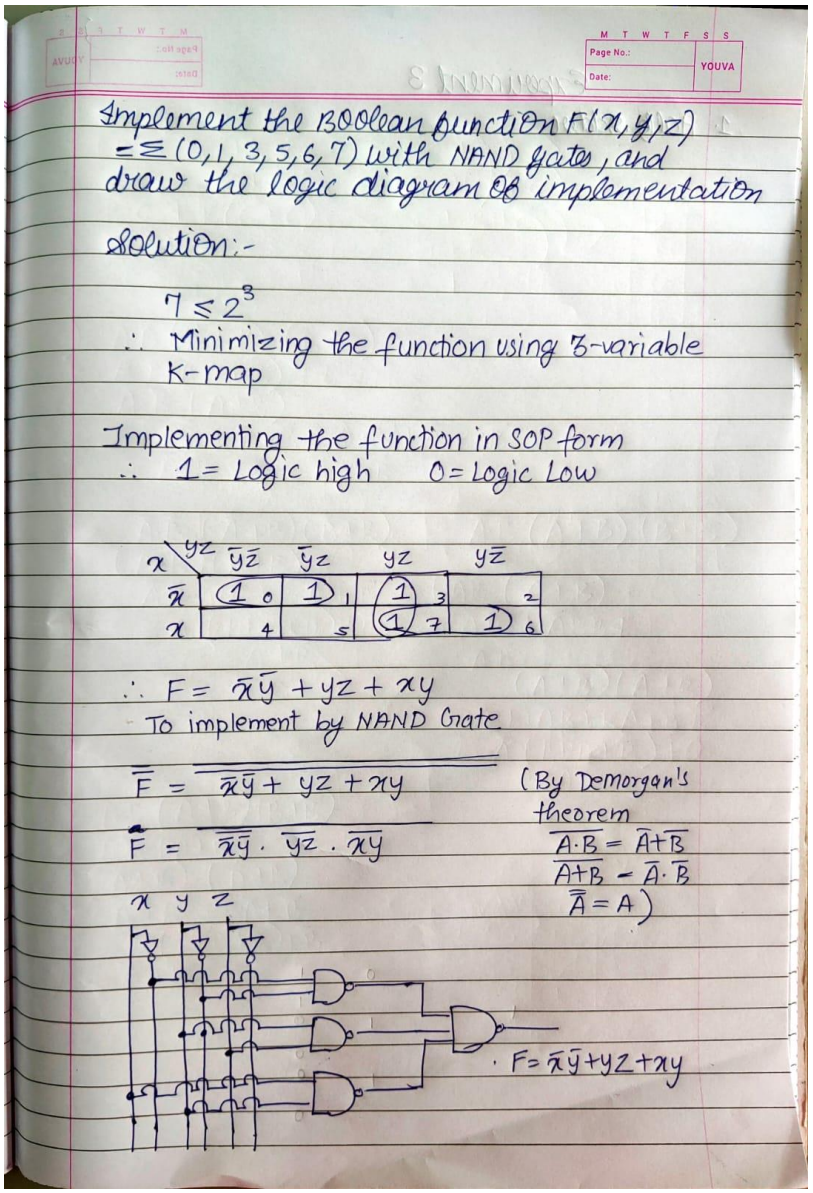
K.J. Somaiya College of Engineering, Vidyavihar (E), Mumbai - 400 077.

NUMERICAL

Implement the Boolean function with NAND Gates and draw the logic diagram of implementation

$F(w,x,y) = \Sigma m(0,1,3,5,6,7)$

SOLUTION



Implement the Boolean function $F(x,y,z) = \Sigma(0,1,3,5,6,7)$ with NAND gates, and draw the logic diagram of implementation

Solution:-

$7 \leq 2^3$

∴ Minimizing the function using 3-variable K-map

Implementing the function in SOP form

∴ 1 = Logic high 0 = Logic Low

x \ yz	$\bar{y}\bar{z}$	$\bar{y}z$	yz	$y\bar{z}$
$\bar{x}$	1 (0)	1 (1)	1 (3)	(2)
x	(4)	(5)	1 (7)	1 (6)

∴ $F = \bar{x}\bar{y} + yz + xy$

To implement by NAND Gate

$\overline{\overline{F}} = \overline{\bar{x}\bar{y} + yz + xy}$

$F = \overline{\bar{x}\bar{y} \cdot \overline{yz} \cdot \overline{xy}}$

(By Demorgan's theorem
$\overline{A \cdot B} = \bar{A} + \bar{B}$
$\overline{A + B} = \bar{A} \cdot \bar{B}$
$\bar{\bar{A}} = A$)